Introduction aux Base de Données

Safa BETTOUMI ZOGHLAMI

September 4, 2026

Introduction (1/2)

Une base de données informatique est une collection d'informations organisées afin d'être facilement consultables, gérables et mises à jour.

Les bases de données sont utilisées pour :

- **Stocker**.
- **Organiser** .
- **Analyser des données**.

Introduction (2/2)

- Les entreprises gèrent des volumes de données très grands:
  - Giga, Terra, Péta-octets.
  - Numériques, Textuelles, Multi-média (images, films,...).
- Il faut pouvoir facilement:
  - Archiver les données sur mémoires secondaires permanents.
  - Retrouver les données pertinentes à un traitement.
  - Mettre à jour les données variant dans le temps.

$\Rightarrow$ **Première approche: Les fichiers!**

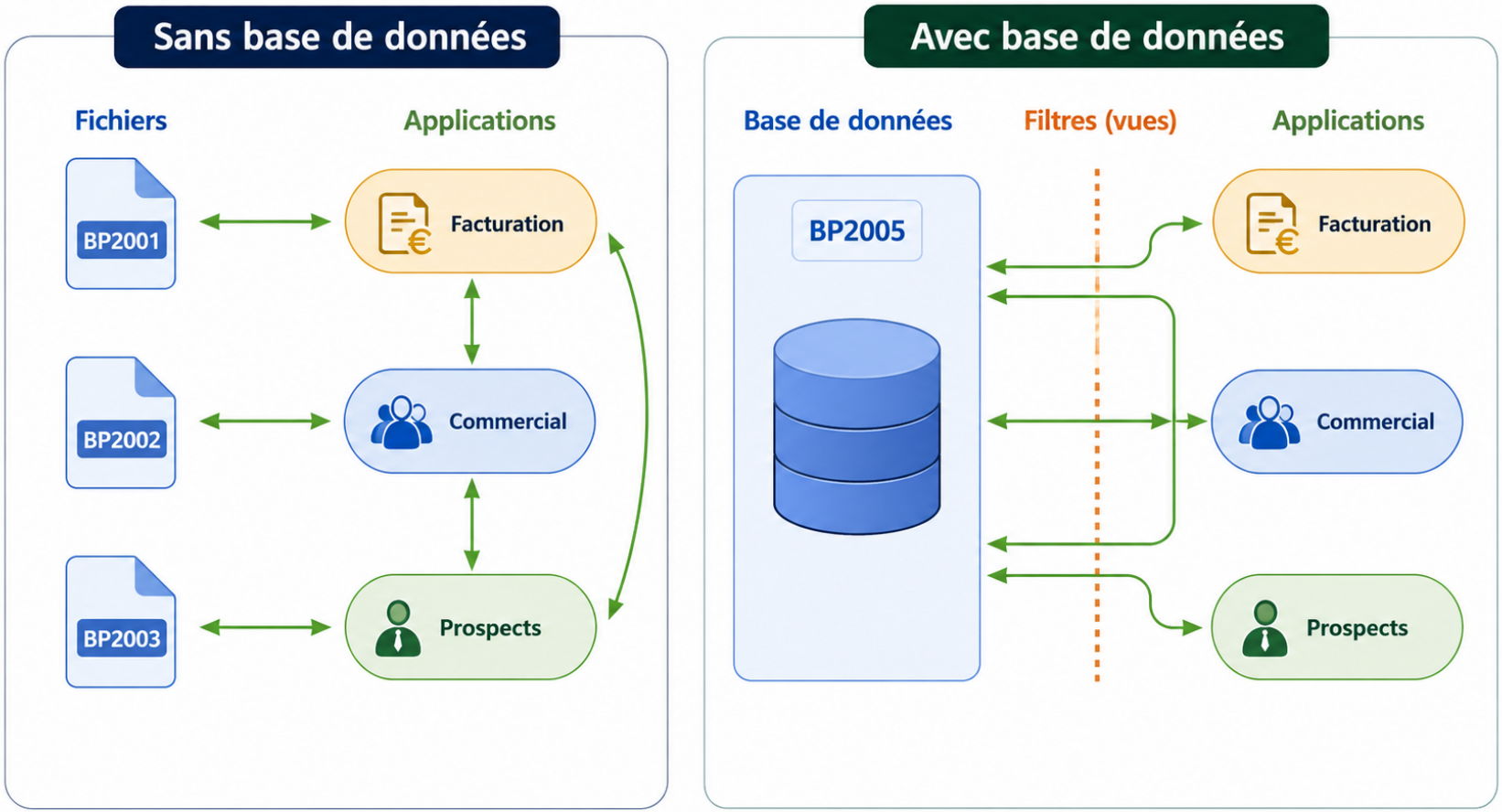
Limites des fichiers

- De telles applications sont:
  - Rigides et contraignantes.
  - Longues et coûteuses à mettre en oeuvre.
  - Manque de sécurité.
- Les données associées sont:
  - Mal définies et mal désignées.
  - Un accès aux données nécessite un programme.
  - Redondantes, incohérentes et peu fiables.
  - Peu accessibles de manière ponctuelle.
  - Le modèle des données est intégré dans les programmes.
  - Absence de contrôle pour l'accès et la manipulation des données.

$\Rightarrow$ **Solution: Evolution vers les BD**

L'approche "Bases de données"?

- Une information n'est stockée qu'une seule fois, une seule base pour toutes les applications, mais chaque application ne voit que celle doit voir (contôle par les filtres ou les vues).

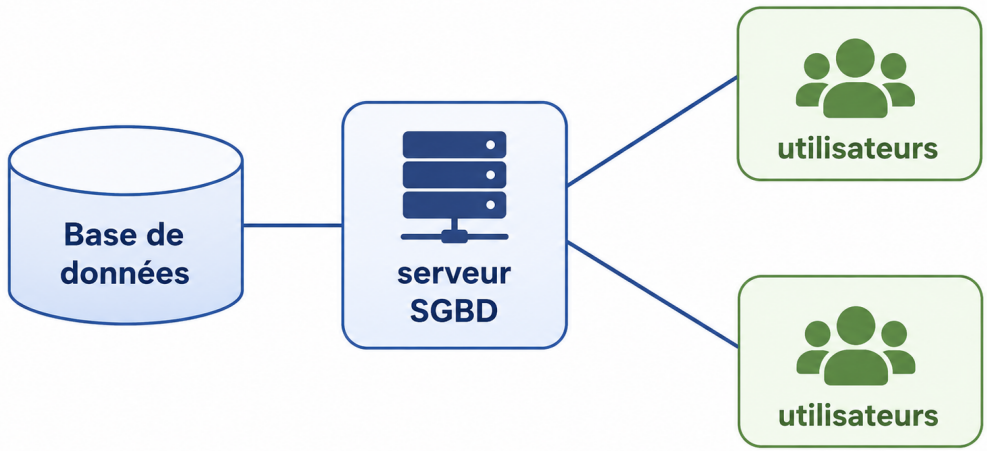


L'approche "Bases de données"?

- **Les données sont totalement indépendantes des programmes.**
- **On peut créer plusieurs vues et des données virtuelles ce qui permet l'adaptation de la présentation des données selon l'utilisateur.**
- **Limiter les accès aux utilisateurs (privilèges, rôles, etc) .**
- **Sécuriser les données (Authentification, droit d'accès, etc).**
- **Etc.**

L'approche "Bases de données"?

- Le concept de base de données comprend essentiellement deux aspects:
  - Un ensemble **structuré d'information.**
  - Un composant logiciel appelé **système de gestion de base de données (SGBD)** et qui permet l'utilisation de ces données simultanément par plusieurs utilisateurs et programmes.



Système de gestion de base de données (SGBD)

- Un SGBD est un logiciel permettant de :
  - Définir et manipuler les données.
  - Garantir l'intégrité et la cohérence des données *(données valides et respect des règles définies).*
  - Gérer la sécurité et les droits d'accès.
  - Gérer les accès concurrents *(accès simultanés de plusieurs utilisateurs).*
  - Assurer la résistance aux pannes et la récupération des données.
  - Assurer l'indépendance des données :
    - **Indépendance physique :** modifier le mode de stockage physique sans modifier les applications.
    - **Indépendance logique :** modifier l'organisation logique des données sans modifier les applications.

Le marché SGBD

Le DB-Engines Ranking classe les systèmes de gestion de base de données en fonction de leur popularité. Le classement est mis à jour mensuellement.

Rank			DBMS	Database Model
Sep 2026	Aug 2026	Sep 2025		
1.	1.	1.	Oracle	Relational, Multi-model
2.	2.	2.	MySQL	Relational, Multi-model
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model
4.	4.	4.	PostgreSQL	Relational, Multi-model
5.	5.	5.	MongoDB	Document, Multi-model
6.	6.	6.	Snowflake	Relational
7.	7.	↑ 9.	Databricks	Multi-model
8.	8.	↓ 7.	Redis	Key-value, Multi-model
9.	9.	↓ 8.	IBM Db2	Relational, Multi-model
10.	↑ 11.	↑ 11.	SQLite	Relational

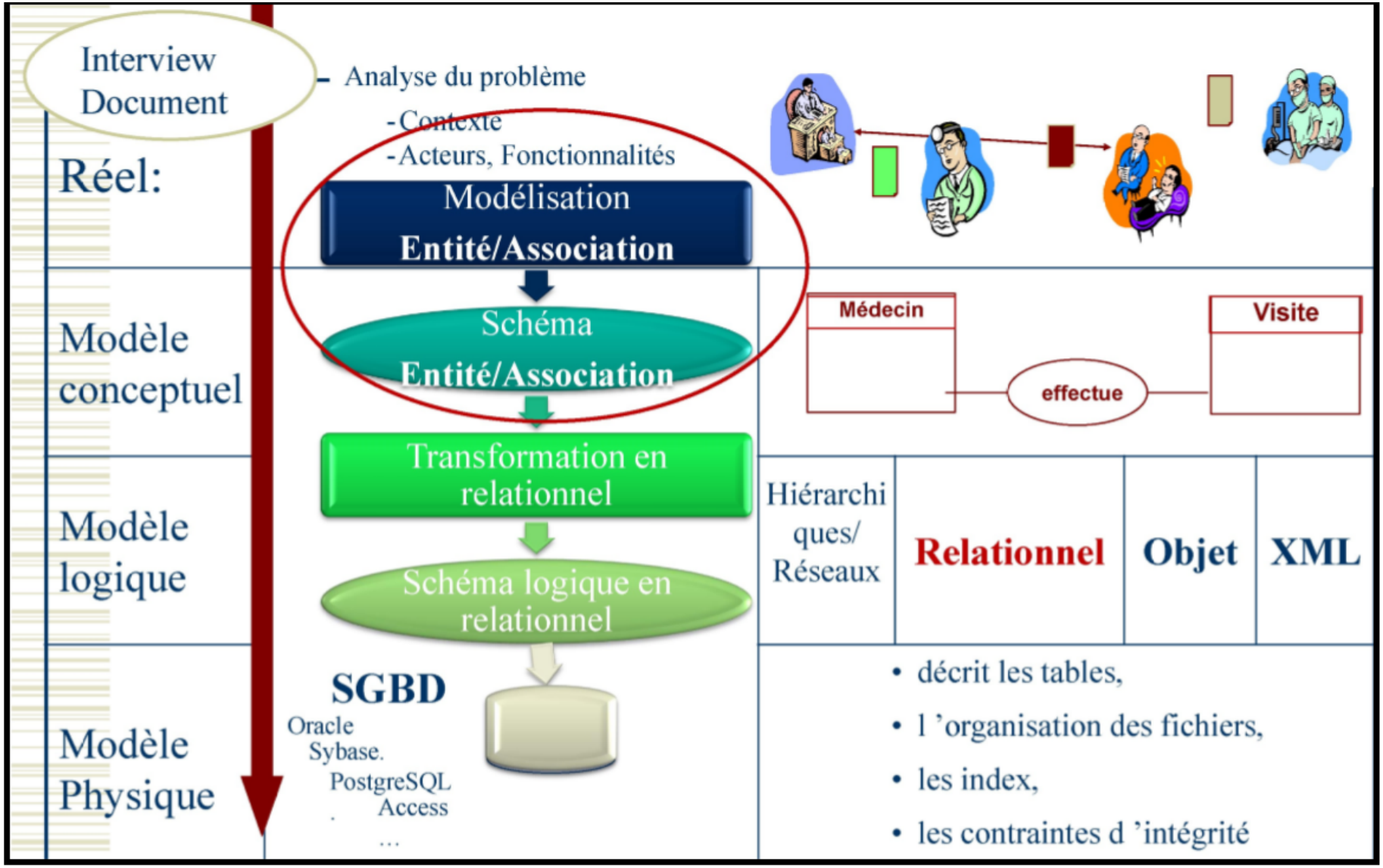
Exemples d'application

- Gestion des personnels, des étudiants, des cours et des inscriptions d'une université.
- Système de réservation de billets d'avion.
- Gestion des comptes clients d'une banque.
- Gestion des commandes d'Amazon.
- Gestion des Jeux olympiques.
- Gestion des projets dans une entreprise.
- Gestion du catalogue d'une bibliothèque.

Modèles de bases de données

- **Modélisation :** processus permettant de représenter et d'organiser les données d'un domaine.
- Un **modèle de données** définit la manière dont les données sont **structurées et reliées entre elles**.
- Principaux modèles :
  - **Modèle hiérarchique :** données organisées sous forme d'arbre (relations parent-enfant).
  - **Modèle relationnel :** les données sont organisées sous forme de tables (relations) composées de lignes (enregistrements) et de colonnes (attributs). Les tables peuvent être reliées entre elles grâce à des clés.
  - **Modèles NoSQL :** modèles non relationnels adaptés notamment à des données variées ou distribuées.

Étapes de construction d'une base de données relationnelle



Modèle conceptuel de données (MCD)

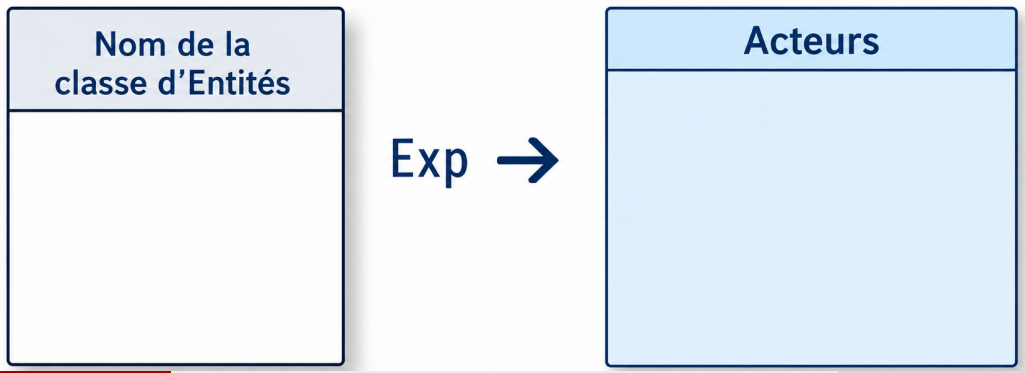
- Le **Modèle Conceptuel de Données (MCD)** est une représentation **abstraite et structurée** des données d'un domaine.
- Il permet de représenter les **informations importantes** du domaine ainsi que les liens entre elles.
- Il est construit **indépendamment de toute considération technique** : SGBD, stockage, langage, etc.
- Le modèle **Entité-Association** est un formalisme graphique permettant de construire et de représenter le MCD.

Modèle Entité-Association

- Le **modèle Entité-Association** est un formalisme graphique permettant de représenter **conceptuellement les données d'un domaine**.
- Il permet de construire un **diagramme Entité-Association** décrivant:
  - les **entités** du domaine ;
  - les **associations** entre les entités ;
  - les **attributs** qui les caractérisent ;
  - les **identifiants** des entités ;
  - les **cardinalités** des associations.
- **Mots clés :** Entité, Association, Attribut, Identifiant, Cardinalité.

Entité VS Classe d'entités

- **Une entité**: est un objet identifiable et nommable du problème, un événement, un lieu, une personne ...
  - Exemple: le cinéma "ABC", le cinéma "AFRICA", l'acteur "Denzel Washington", le film "Bonheur est dans le pré", le film "The Equalizer".
  - C'est une instance de la classe d'entité.
- **Classe d'entités**: c'est un regroupement d'entités de même nature (un ensemble d'entités).
  - Exemple: Exemple: Cinema (avec les propriétés communes à tous les cinémas), Acteurs, Films, etc.
  - Formalisme de classe d'entité : représentation graphique

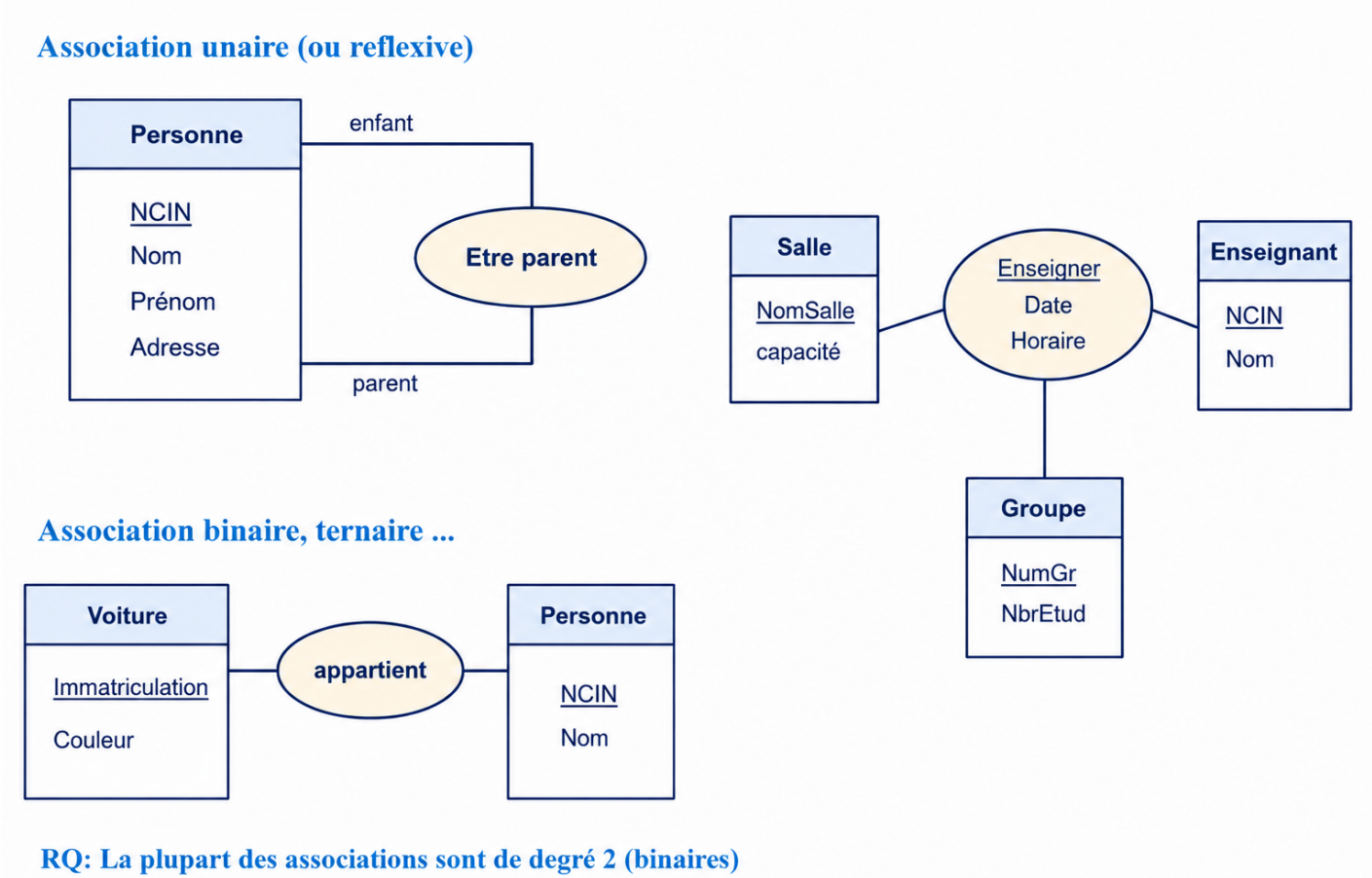


Association

- **Classe d'association :** c'est un lien entre deux ou plusieurs classes d'entités.
  - Exemples :
    - Jouer (entre FILMS et ACTEURS).
    - Afficher (entre CINÉMA et FILMS).
    - Réaliser (entre RÉALISATEURS et FILMS).
    - Chaque entité joue un rôle dans une association.
- **Une association :** c'est un lien entre deux ou plusieurs entités.
  - Exemples:
    - Woody allen a joué dans "Coups de feu sur brodway".
    - Jérome a écrit l'ouvrage "Bases de données".

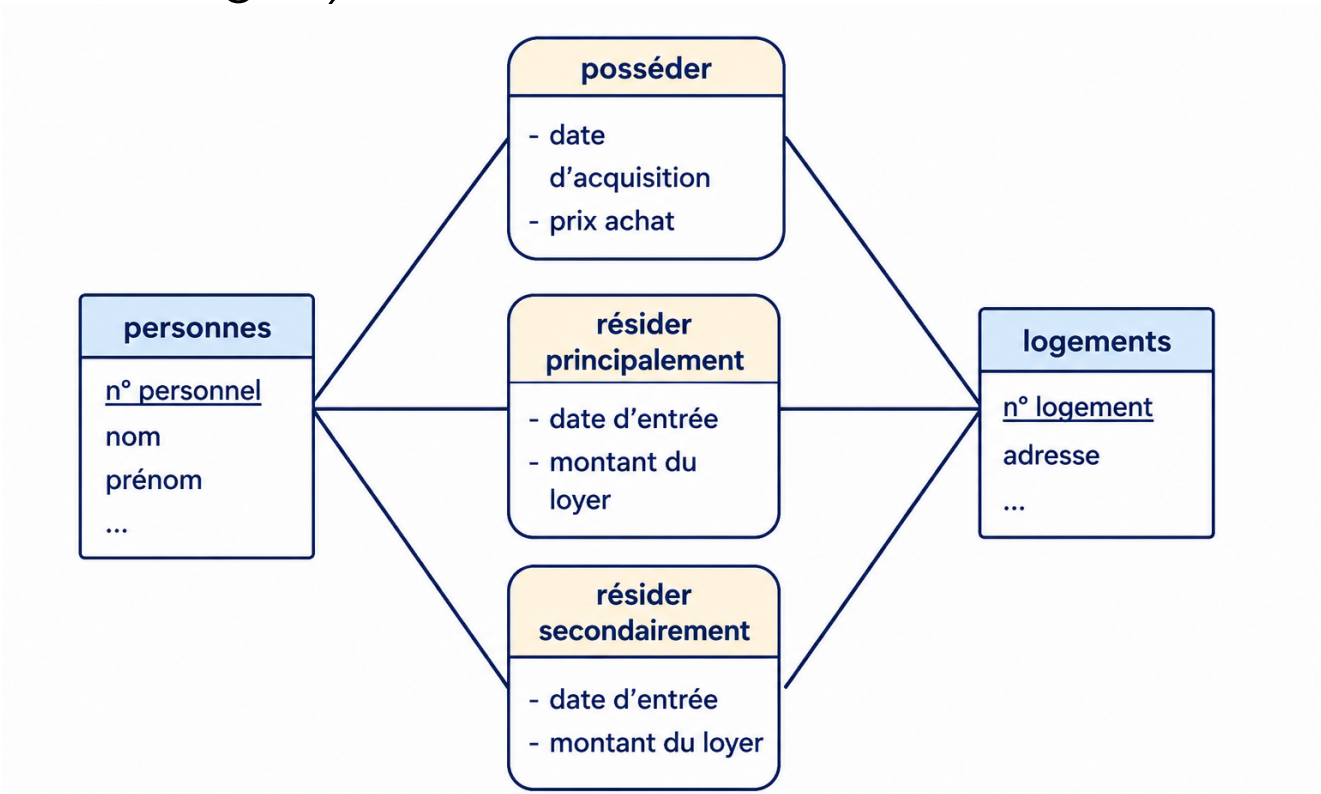
Association

- Le degré d'une association
  - C'est le nombre n de classes d'entités participant à cette association (si degré=n, on dit association n-aire).
  - Exemples:



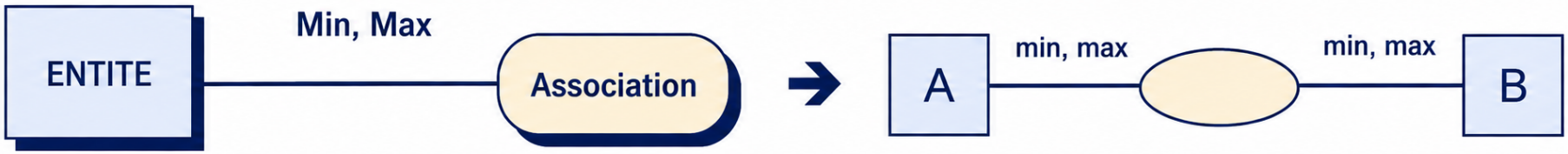
Association

- Association plurielle
  - **Remarque : On peut avoir plusieurs associations entre les mêmes entités**.
  - Deux mêmes entités peuvent être plusieurs fois en association (c'est le cas sur la figure).



Association

- **Cardinalité:**
- Les cardinalités d'une entité dans une association expriment le nombre de fois qu'une occurrence de cette entité peut être impliquée dans une occurrence de l'association, au minimum et au maximum.
  - Elles traduisent des règles de gestion.
    - Ces règles expriment des contraintes sur le modèle.
    - Les valeurs possibles sont (0,1), (1,1), (0,N), (1,N).



Association

- **Cardinalité Minimale (0,1):**
  - Combien de fois au minimum une occurrence de l'entité participe à l'association?
- Exemple
  - Pour la cardinalité minimale entre "client et commander", il faut se poser la question:
    - Pour un client donné, combien de fois au minimum il commande?
    - Ou encore mieux: est il obligatoire qu'un client effectue une commande de produit?
  - $\Rightarrow$ Cela dépend des RÈGLES DE GESTION de l'entreprise.
- **Cas 1: Si la règle de gestion est " tout client doit passer au moins une commande sinon ce n'est pas un client" on met la cardinalité minimale à 1.**

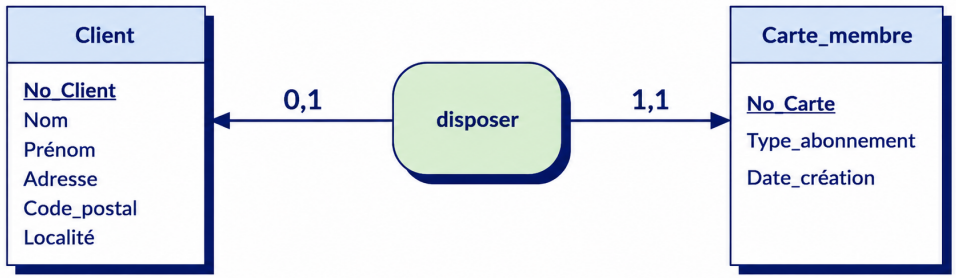
Association

- **Cardinalité Minimale(0,1):**
  - **Cas 2: L'entreprise veut aussi mémoriser les clients potentiels (prospects), qui n'ont encore rien commandé.**
  - **Dans ce cas, un client peut très bien ne pas avoir encore commandé, et on met la cardinalité minimale à 0.**

Valeur	Définition
0	Une occurrence de l'entité peut exister sans participer à l'association
1	Une occurrence de l'entité participe nécessairement au moins une fois à une occurrence d'association

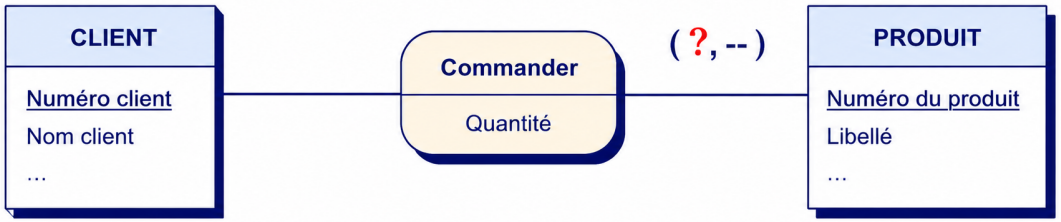
Association

- Les **cardinalités minimales** permettent de définir la dépendance d'une entité vis-à-vis d'une association :
  - On dit qu'une entité est indépendante par rapport à une association lorsque sa cardinalité minimale vaut 0, et dépendante par rapport à une relation lorsque sa cardinalité minimale vaut 1.
  - Exemple:



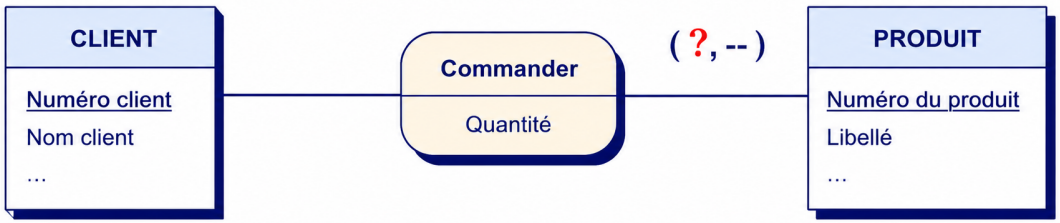
- Un client peut donc exister sans avoir de carte de membre, alors qu'une carte de membre ne peut pas exister sans être associée à un client.

Activité 1



- **Question** : Est-il obligatoire qu'un produit ait été commandé pour être mémorisé dans la base de données ? Ou bien, peut-il exister dans la base de données des produits qui n'ont jamais été commandés ?
- Formulez deux hypothèses de règles de gestion concernant ce lien et déterminez les cardinalités minimales correspondantes.

Réponse



- **H1 :** L'entreprise peut proposer de nouveaux produits, par exemple dans un catalogue, et les enregistrer dans sa base de données même s'ils n'ont encore jamais été commandés. $\Rightarrow$ **Cardinalité minimale : 0.**
- **H2 :** Tout produit enregistré dans la base de données est nécessairement commandé. Par exemple, l'entreprise ne propose pas de catalogue et fabrique uniquement à la commande. $\Rightarrow$ **Cardinalité minimale : 1.**

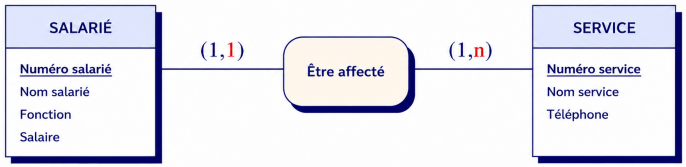
Association

- **Cardinalité maximale (1 ou N) :**
  - Cette cardinalité répond aux questions suivantes :
    - La participation d'une occurrence doit-elle être unique ?
    - Ou bien peut-elle être multiple ?
    - Combien de fois au maximum une occurrence peut-elle participer à l'association ?
  - Si la réponse est "au plus une fois" (participation unique) :
    - La cardinalité maximale vaut 1.
  - Si la réponse est "plusieurs fois" (participation multiple) :
    - La cardinalité maximale vaut N.

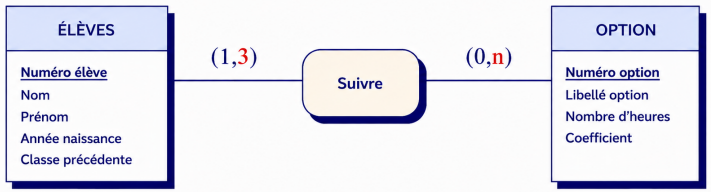
Valeur	Définition
1	Une occurrence de l'entité participe au plus une fois
N	Une occurrence de l'entité peut participer plusieurs fois

Activité 2

- **RG (règles de gestion) :**
  - Un salarié est affecté à un seul service au maximum.
  - Un service peut regrouper plusieurs salariés.



- Une cardinalité maximale peut, dans certains cas, avoir une valeur limitée autre que 1.
  - **Exemple – RG :** Un élève peut suivre au maximum trois options.



Typologie d'une association

- **1-1**

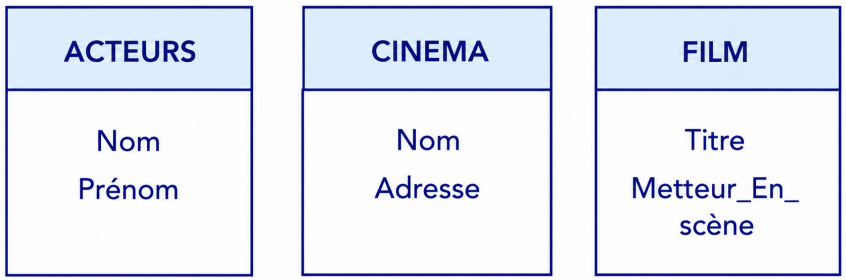
- **1-N**

- **N-M**

Remarque : La typologie d'une association est importante pour déterminer les règles de passage au modèle relationnel.

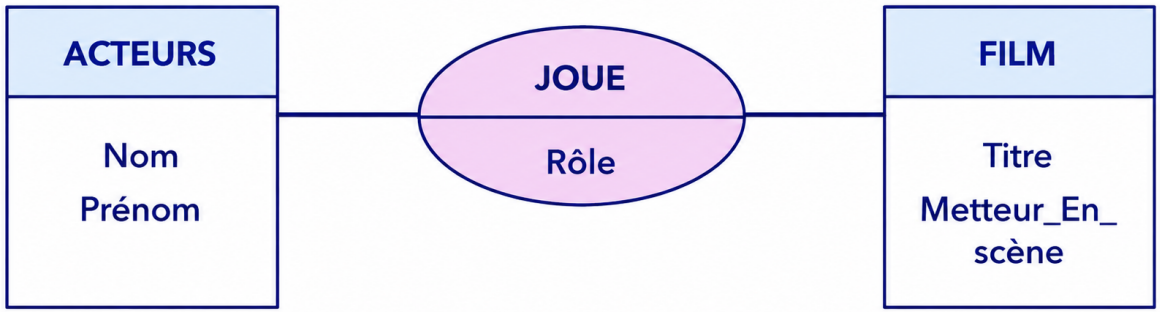
Attribut

- Une propriété ou un attribut est une donnée élémentaire qui caractérise une **entité ou une association**.
  - Exemples :
    - Pour la classe d'entités ACTEURS : Nom, prénom, etc.
    - Pour la classe d'entités FILMS : Titre, metteur en scène, etc.
    - Pour la classe d'entités CINEMA : Nom, adresse, etc.
  - Formalisme graphique :



Attribut

- **Pour les associations:**
  - **Les propriétés** que l'on met dans une association doivent obligatoirement relier les entités.
  - Exemple: le rôle d'un acteur



  - Remarque : cette association est dite association porteuse.

Attribut

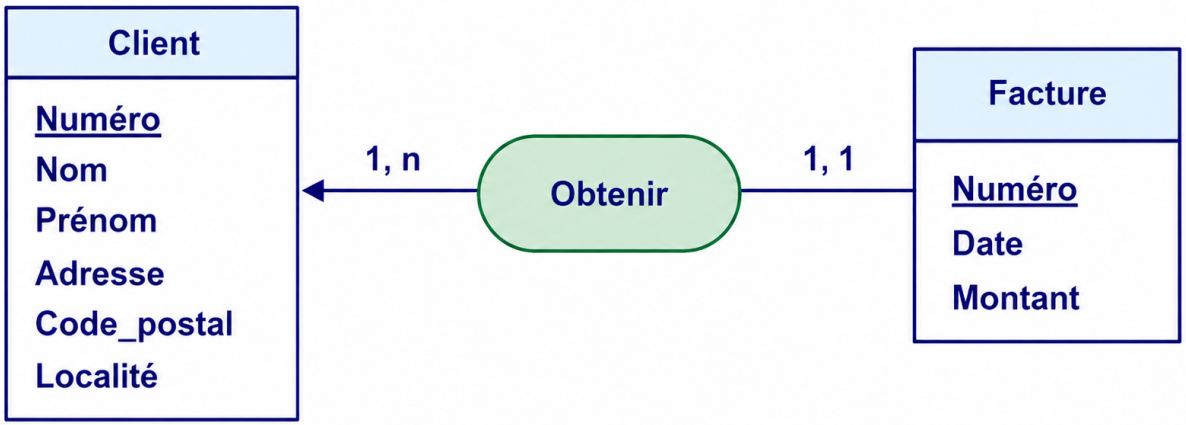
- **Simple / composé**
  - Simple (atomique ou non décomposable) : attribut ayant une valeur qui ne peut pas être décomposée (exemples : jour, prénom).
  - Composé : attribut pouvant être décomposé en plusieurs attributs (exemples : date $\rightarrow$ jour, mois, année ; adresse $\rightarrow$ rue, ville, code postal).
- **Monovalué / multivalué**
  - Monovalué : une seule valeur par occurrence (exemples : date de naissance, numéro de sécurité sociale).
  - Multivalué : plusieurs valeurs possibles pour une même occurrence (exemples : prénoms, numéros de téléphone).
- **Obligatoire / facultatif**
  - Obligatoire : une valeur doit être renseignée pour chaque occurrence (exemple : nom).
  - Facultatif : peut ne pas avoir de valeur (exemples : téléphone, deuxième prénom).

Identifiant ou clé

- Un **identifiant (ou clé)** est un attribut qui permet d'**identifier de manière unique** chaque occurrence d'une entité.
  - Exemple : le NSS pour une personne.
- Un identifiant peut être constitué de **plusieurs attributs** : on parle alors de **clé composée**.
  - Exemple :
    - [Numéro, Rue, Ville] pour une maison.
- Dans un modèle E/A, l'identifiant est **souligné**.
- **Choix de l'identifiant :**
  - On privilégie un identifiant **simple, court et stable** (exemples : numéro d'étudiant, numéro de commande).
  - Si aucun attribut naturel ne convient, on peut créer un **identifiant artificiel**.

Contrainte d'intégrité fonctionnelle

- Lorsqu'une entité participe à une association avec une cardinalité maximale égale à 1, on parle de **Contrainte d'Intégrité Fonctionnelle (CIF)**.
- Une CIF signifie qu'une occurrence d'une entité **détermine de manière unique** une occurrence de l'autre entité.
  - Exemple : connaissant une facture précise, on peut déterminer avec certitude le client auquel elle appartient.



La relation *Obtenir* représente une **CIF**

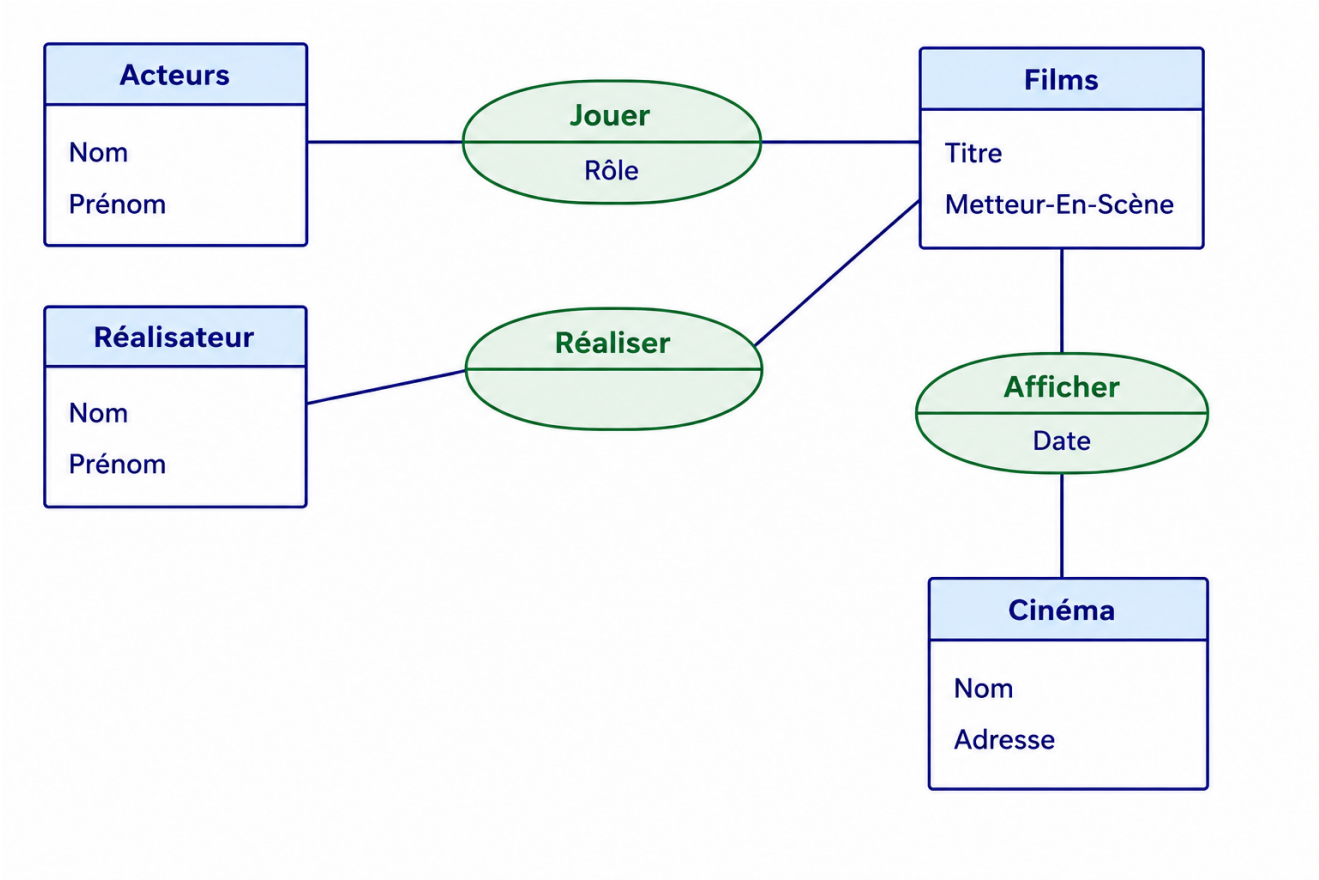
Démarche de conception

- Identifier les **entités** du domaine.
- Identifier les **associations** entre les entités.
- Identifier les **attributs** qui caractérisent les entités et les associations.
- Déterminer les **identifiants** des entités.
- Déterminer les **cardinalités** de chaque association à partir des règles de gestion.
- Identifier les éventuelles **contraintes d'intégrité (CI)**, notamment les CIF.
- **Vérifier et valider** le modèle obtenu avec les règles de gestion.

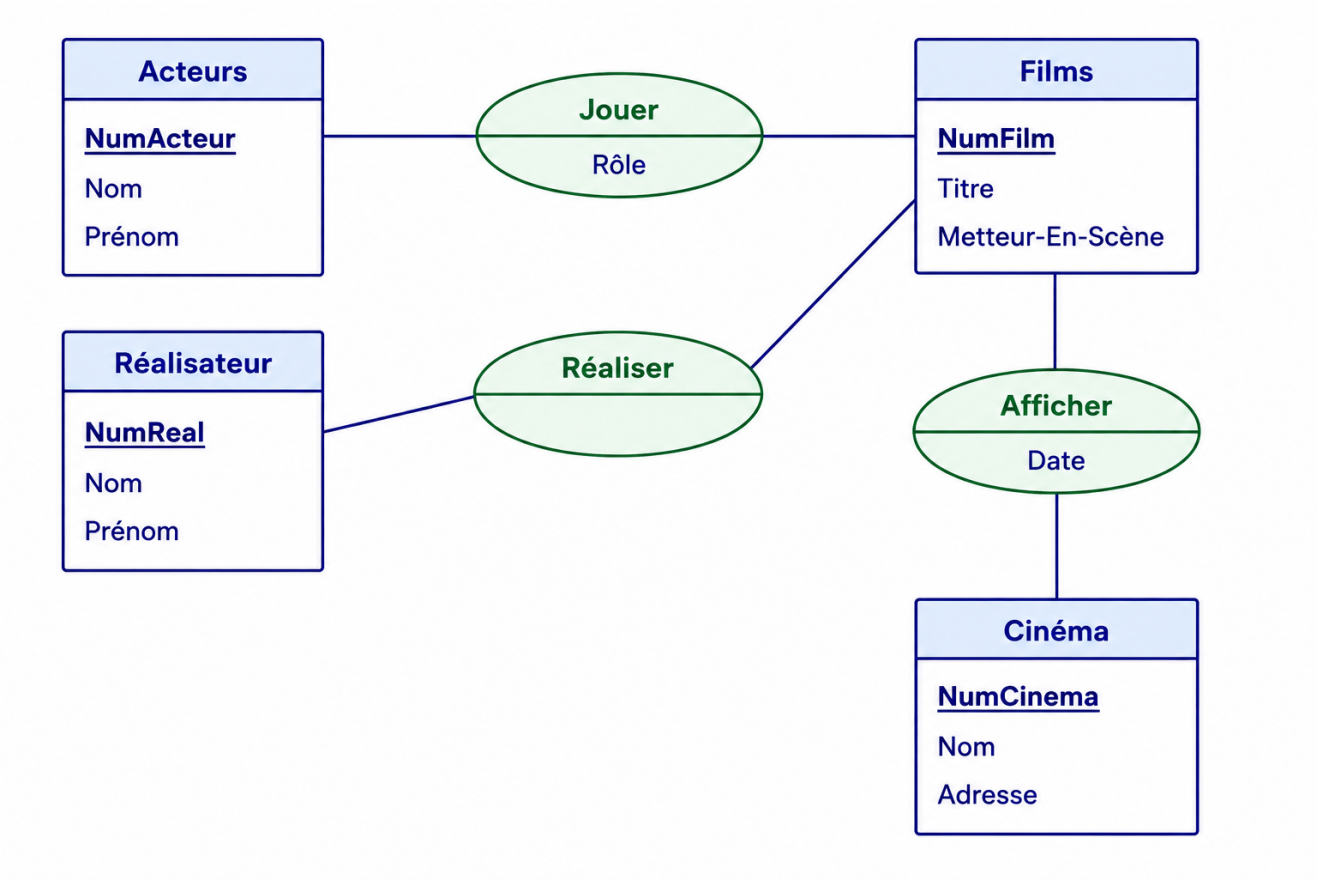
Exemple de schéma global

- **Gestion des films :**
  - Un **acteur** a joué dans **au moins un film** et peut avoir joué dans **plusieurs films**.
  - Un **film** peut ne pas avoir d'acteurs, par exemple s'il s'agit d'un film documentaire.
  - Un **film** peut avoir **plusieurs acteurs**.
  - Un **film** a **un et un seul réalisateur**.
  - Un **film** est projeté à une **date précise**.

Exemple de schéma global (E/A)

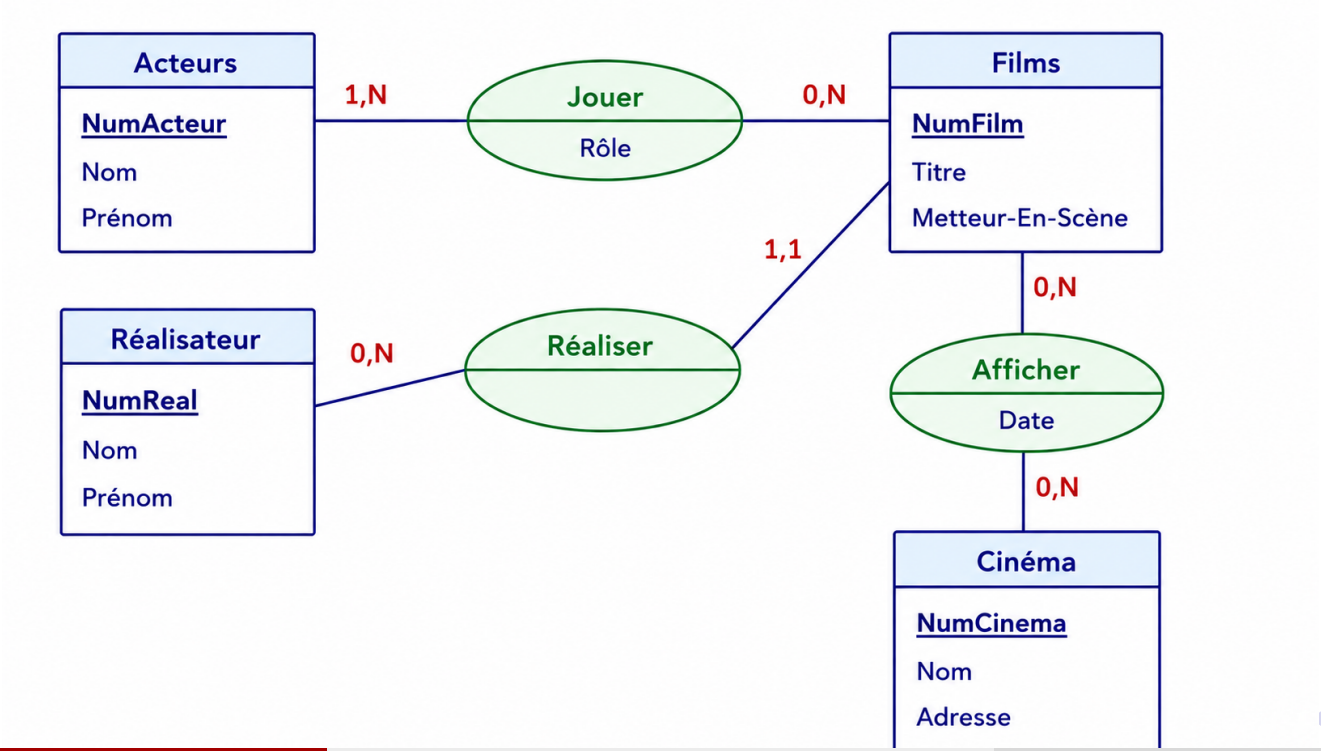


Exemple de schéma global (E/A)



Exemple de schéma global (E/A)

- Gestion de films:
  - Un acteur a joué dans au moins un film. Un acteur peut avoir joué dans plusieurs films.
  - Un film n'ayant pas d'acteurs, possible si c'est un film documentaire
  - Un film peut avoir plusieurs acteurs
  - Un film a au moins un réalisitaeur
  - Il y a au plus un réalisateur.
  - Un film est affiché à une date précise.



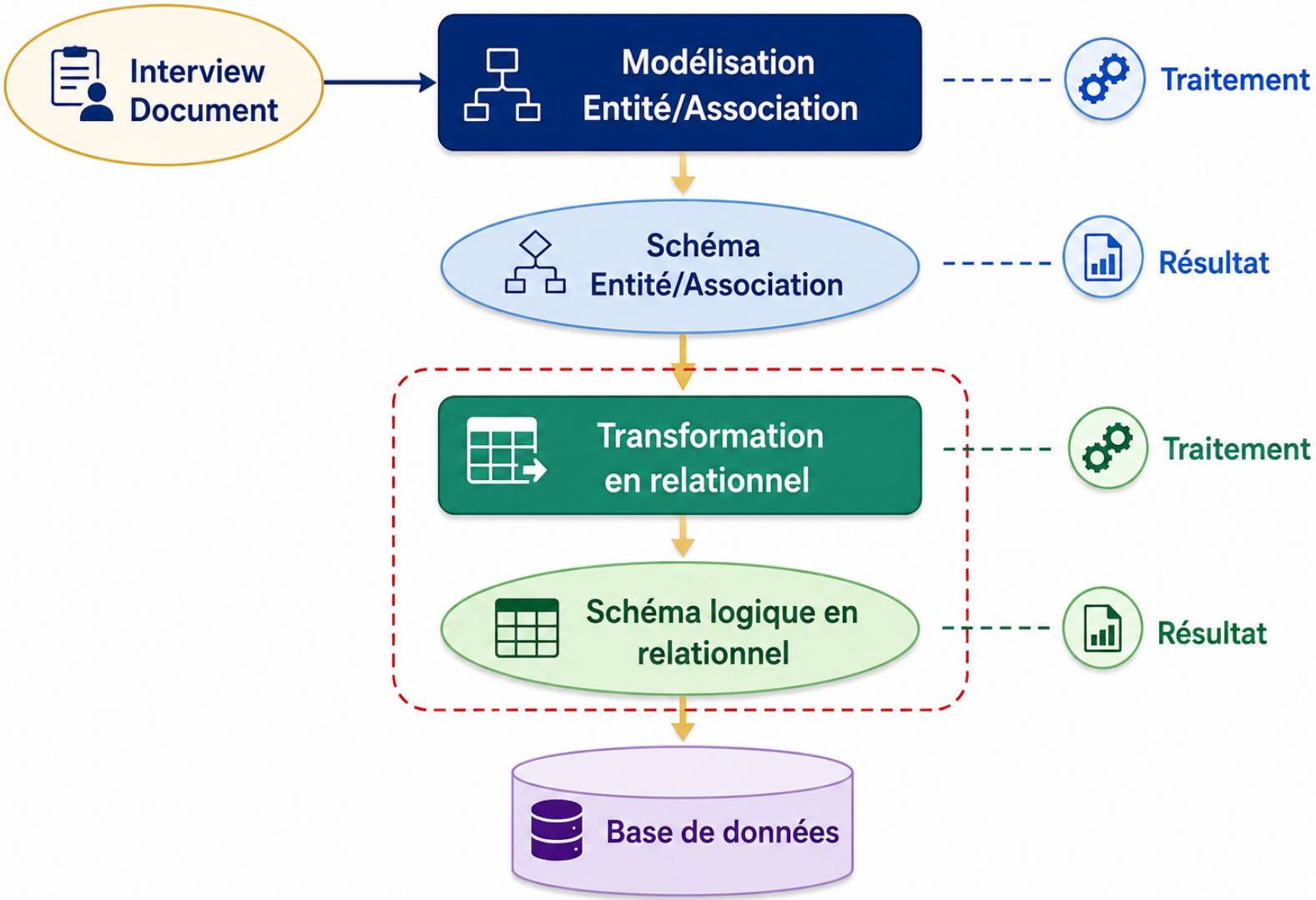
Vérification et validation d'un modèle E/A

- Toute entité doit posséder un **identifiant unique et non nul**.
- Chaque attribut doit être **rattaché à l'entité ou à l'association** qu'il caractérise réellement.
- **Éviter les attributs calculés :**
  - Un attribut pouvant être déduit d'autres attributs ne doit généralement pas être mémorisé.
  - Exemple : Âge calculé à partir de la date de naissance.
- **Éviter les attributs complexes :**
  - Décomposer l'attribut lorsque ses composants doivent être manipulés séparément.
  - Exemple : Adresse $\rightarrow$ Numéro, Rue, Ville, Code postal.

Vérification et validation d'un modèle E/A

- **Éviter les attributs multivalués :**
  - Un attribut ne doit pas contenir plusieurs valeurs pour une même occurrence.
  - Exemple : un étudiant peut pratiquer plusieurs langues.
    $\Rightarrow$ Solution : créer une entité **Langue** associée à l'entité **Étudiant**.
- **Vérifier les cardinalités :**
  - Elles doivent être cohérentes avec les **règles de gestion**.
- **Éviter les redondances :**
  - Une même information ne doit pas être représentée plusieurs fois inutilement dans le modèle.
- **Vérifier la complétude du modèle :**
  - Toutes les données et règles de gestion nécessaires doivent pouvoir être représentées.

Démarche de construction d'une BD



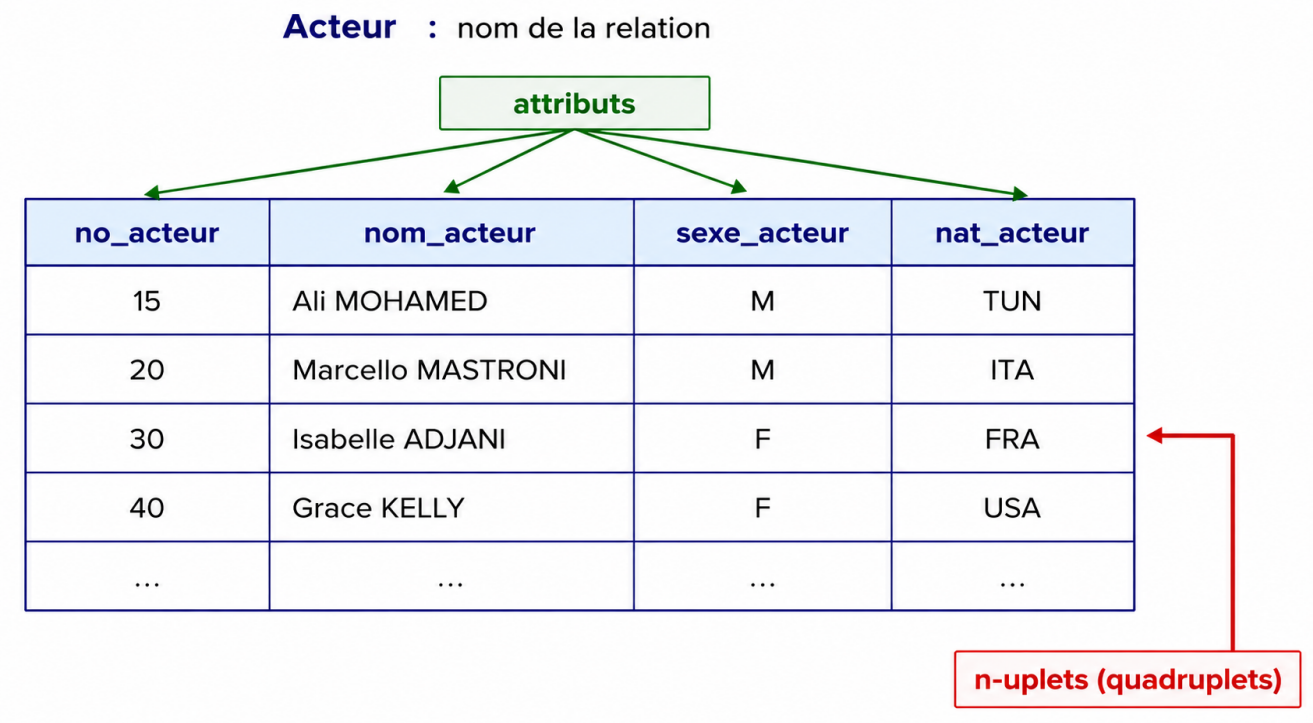
Le modèle relationnel

- Le **modèle relationnel** organise les données sous forme de **tables**, appelées **relations**.
- Une table est composée de :
  - **Colonnes (attributs) :** elles décrivent les propriétés des données. Chaque attribut possède un **nom** et un **domaine** (ensemble des valeurs qu'il peut prendre).
  - **Lignes (tuples ou n-uplets) :** chaque ligne représente une occurrence enregistrée dans la table.
- Chaque case de la table contient la **valeur d'un attribut** pour un tuple donné.

Modèle à deux concepts

- Modèle à deux concepts :
  - Relation ⇒ Table
  - Attribut ⇒ colonne

Exemple:



no_acteur	nom_acteur	sexe_acteur	nat_acteur
15	Ali MOHAMED	M	TUN
20	Marcello MASTRONI	M	ITA
30	Isabelle ADJANI	F	FRA
40	Grace KELLY	F	USA
...	...	...	...

Structure de base de données

- **Relation:**

Acteur : nom de la relation

attributs

no_acteur	nom_acteur	sexe_acteur	nat_acteur
15	Ali MOHAMED	M	TUN
20	Marcello MASTRONI	M	ITA
30	Isabelle ADJANI	F	FRA
40	Grace KELLY	F	USA
…	…	…	…

n-uplets (quadruplets)

- Une relation a un **nom**.
- Une **colonne** de la table est appelée attribut.
- Une **ligne** de la table est appelée **n-uplets** (tuple en anglais).
- L'**ordre** des lignes et des colonnes n'a pas d'importance.

Degré : Nombre de colonnes (attributs)

Structure d'une base de données

- **Schéma d'une relation**
  - Le schéma d'une relation décrit la **structure de la relation**.
  - Il est caractérisé par :
    - **Un nom** : R.
    - **Une liste d'attributs** : $A_1, A_2, \ldots, A_n$.
    - **Un domaine** associé à chaque attribut : $\mathrm{dom}(A_i) = D_i$.
    - **Une clé primaire** permettant d'identifier de manière unique chaque tuple.
    - Éventuellement, des **clés étrangères** et des **contraintes d'intégrité**.

Notation : $R(A_1 : D_1,\ A_2 : D_2,\ \ldots,\ A_n : D_n)$

Structure de base de données

- **Schéma d'une relation:**

  Notation : $R(A_1 : D_1,\ A_2 : D_2,\ \ldots,\ A_n : D_n)$

Exp : Étudiant (NumEtud ; D1, Nom ; D2, Prénom ; D2, DateNaissance ; D3)

Avec : D1 = {nombres à 6 chiffres}, D2 = {chaînes de 40 caractères} et D3 = {Dates}

Nom de la relation

Clé primaire de la relation (soulignée)

Attributs de la relation (non clé)

Attribut ou champ

Schéma

Population

Tuples, Occurrences ou Enregistrements

Étudiant			
NumEtud	**Nom**	**Prénom**	**Age**
136	Dupont	Lisa	21
211	Bernard	Jean	19
101	François	Élisa	20
245	Martin	Thomas	21

Clé

- **Clé candidate:** sous-ensemble minimal d'attributs qui permet d'identifier chacun des tuples de la relation.
- **Clé primaire:** une des clés candidates, choisie comme identifiant privililégié (par le concepteur de la base).

  Exemple: Etudiant(NumEtud: D1, Nom:D2, Prénom: D2, DateNaissance: D3)
  - 1 clé candidate: numéro de l'étudiant
  - Si on est certain que 2 étudiants n'ont jamais le même nom, nom est une 2ème clé candidate.
- **Clé étrangère**: étant données deux relations R1 et R2, on dit que l'ensemble R1.K d'attributs de R1 est clé étrangère et référence R2 si les attributs R1.K ont les mêmes domaines que les attributs de la clé primaire de la relation R2.
  - Exemple: dans l'association CLIENT lance Commande
  - R2: Client(NClient: entier, Nom:D2, Prénom:D3)
  - R1: Commande(Ncde:entier, DateCde:date, NClient#:entier)

Les règles d'intégrité structurelle

- **Unicité de la clé (Key)**
  - Par définition, une **relation est un ensemble de tuples**.
  - Un ensemble ne contenant pas d'éléments en double, **deux tuples identiques ne peuvent pas apparaître dans une même relation**.
  - Afin d'identifier de manière unique chaque tuple d'une relation, on utilise la notion de **clé**.
  - Une clé est constituée d'un ou de plusieurs attributs permettant d'identifier de manière unique chaque tuple.

Les règles d'intégrité structurelle

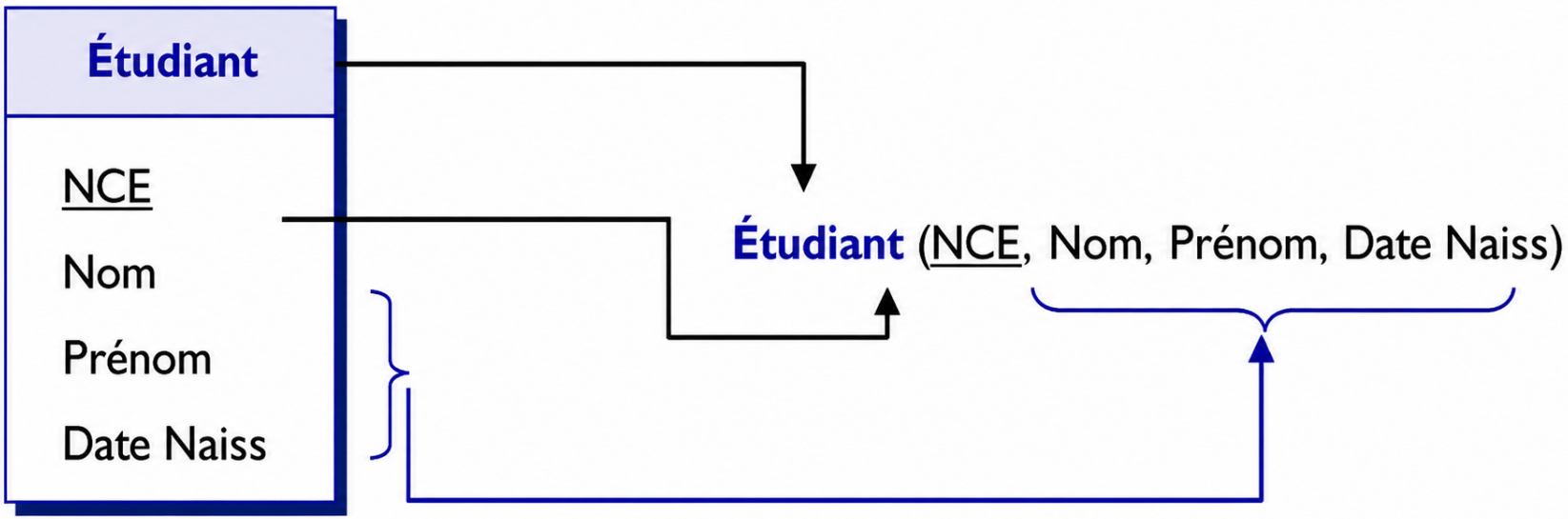
- **Contrainte d'intégrité référentielle**
  - Les contraintes référentielles permettent d'assurer la **cohérence des liens entre les relations**. Elles doivent notamment être respectées lors des opérations d'insertion, de modification ou de suppression.
  - Une contrainte d'intégrité référentielle impose que la valeur d'une **clé étrangère** d'une relation référençante (ex. : R1) corresponde à une valeur de clé dans la relation référencée (ex. : R2).
  - Lors de l'insertion d'un tuple dans la relation référençante R1, il faut donc vérifier que la valeur de la clé étrangère (ex. : R1.NClient) existe dans la relation référencée R2.
  - Exemple : un CLIENT passe une COMMANDE.
    - R2 : Client(NClient : entier, Nom : D2, Prénom : D3)
    - R1 : Commande(NCde : entier, DateCde : date, NClient# : entier)
    - NClient# est une **clé étrangère** qui référence la clé primaire NClient de Client.

Les règles d'intégrité structurelle

- **Valeurs nulles et clés**
  - Lors de l'insertion d'un tuple dans une relation, la valeur d'un attribut peut être **inconnue** ou **non applicable**.
  - On utilise alors une valeur conventionnelle appelée **valeur nulle (NULL)**.
    - NULL représente l'absence de valeur, par exemple une information inconnue ou non applicable.
  - **Contrainte d'intégrité de la clé primaire :**
    - Toute relation possède une **clé primaire**.
    - La valeur de la clé primaire doit être **unique** pour chaque tuple.
    - Aucun attribut participant à la clé primaire ne peut prendre la valeur **NULL**.

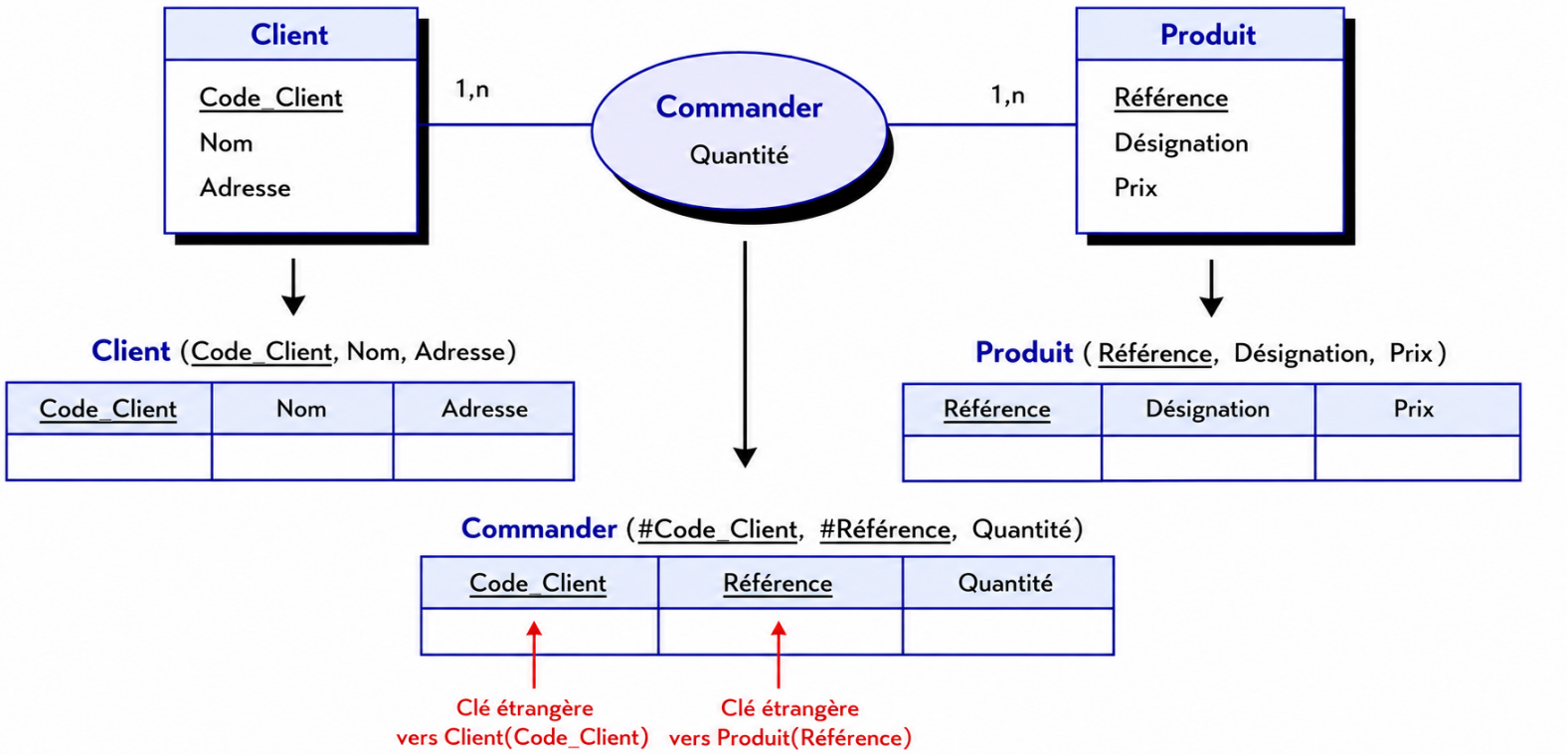
Passage du modèle E/A au modèle relationnel

- **Règle 1: Traduction des entités**
  - Toute entité devient une relation dans laquelle les propriétés deviennent des attributs.
  - L'identifiant de l'entité constitue alors la clé primaire de relation $\Rightarrow$ attribut souligné.



Passage du modèle E/A au modèle relationnel

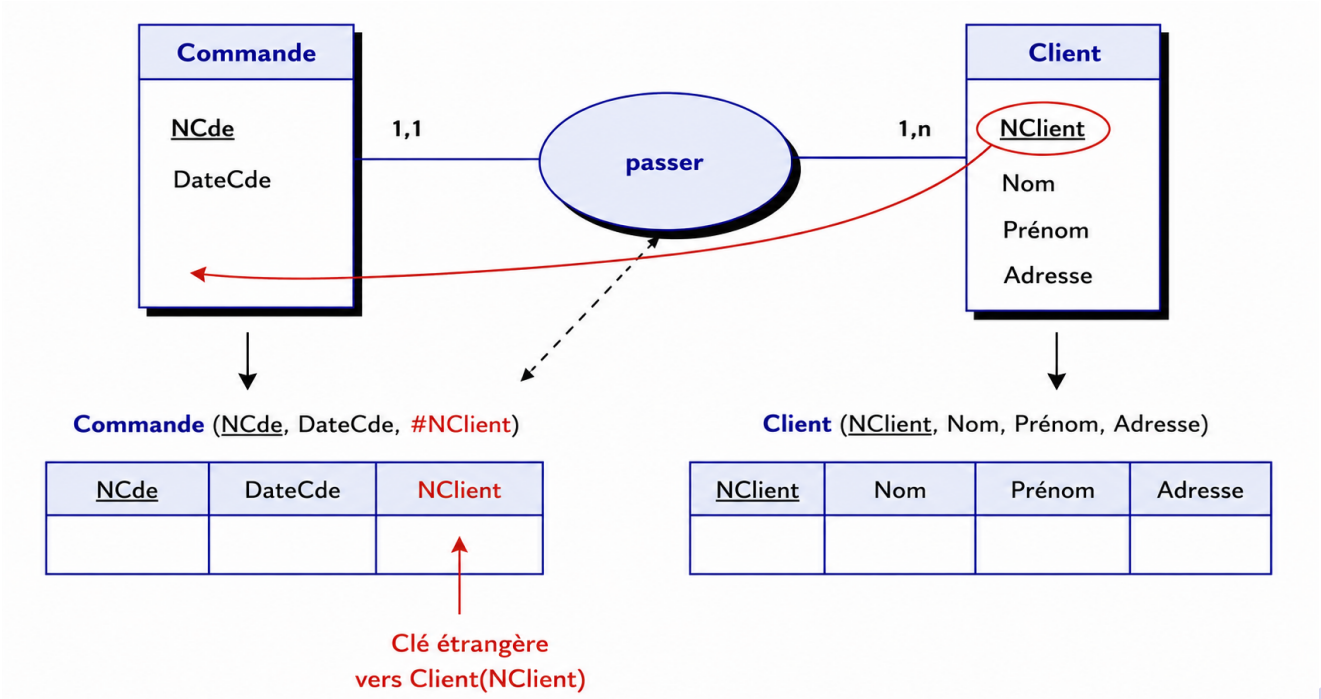
- **Règle 2: Traduction des relations N-M**
  - Une association entre deux entités de type N-M est traduite par une relation supplémentaire (parfois appelée table de jointure) dont la clé primaire est composée de deux clés étrangères référençant les clés primaires des deux entités en association. Les propriétés de l'association deviennent des attributs de cette relation.



Passage du modèle E/A au modèle relationnel

- **Règle 3: Traduction des associations 1-N**
- Cas d'ossociation (*,1)...(*,N):

  Dans le cas de deux entités reliées par une association de type 1,1-*,N, on ajoute une clé étrangère dans la relation côté 1,1, référençant la clé primaire de la relation côté 0,N ou 1,N. Les propriétés de l'association glissent vers la relation côté 1 ou 1,1.

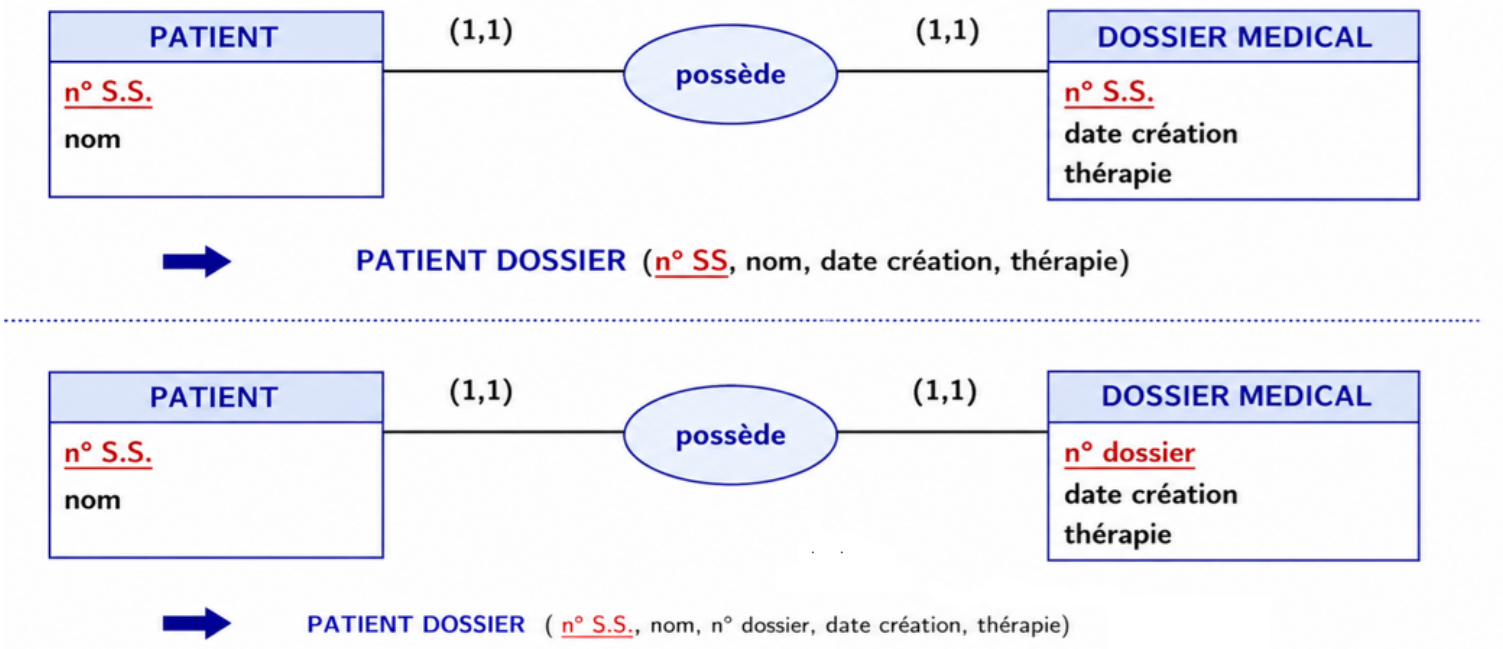


Passage du modèle E/A au modèle relationnel

- **Règle 4 : Traduction des associations 1–1 (1/4)**
  - Dans une association **1–1**, chaque occurrence d'une entité est associée à **au plus une occurrence** de l'autre entité.
  - Deux solutions de traduction sont possibles :
    - **Solution 1 : conserver les deux relations**
      On introduit la **clé primaire de l'une comme clé étrangère** dans l'autre.
    - **Solution 2 : fusionner les deux entités**
      Les deux entités sont regroupées dans **une seule relation**, lorsque cela est pertinent.
  - **Le choix dépend principalement :**
    - des **cardinalités minimales** $(0,1)$ ou $(1,1)$;
    - de la **sémantique et de l'utilisation des données**.

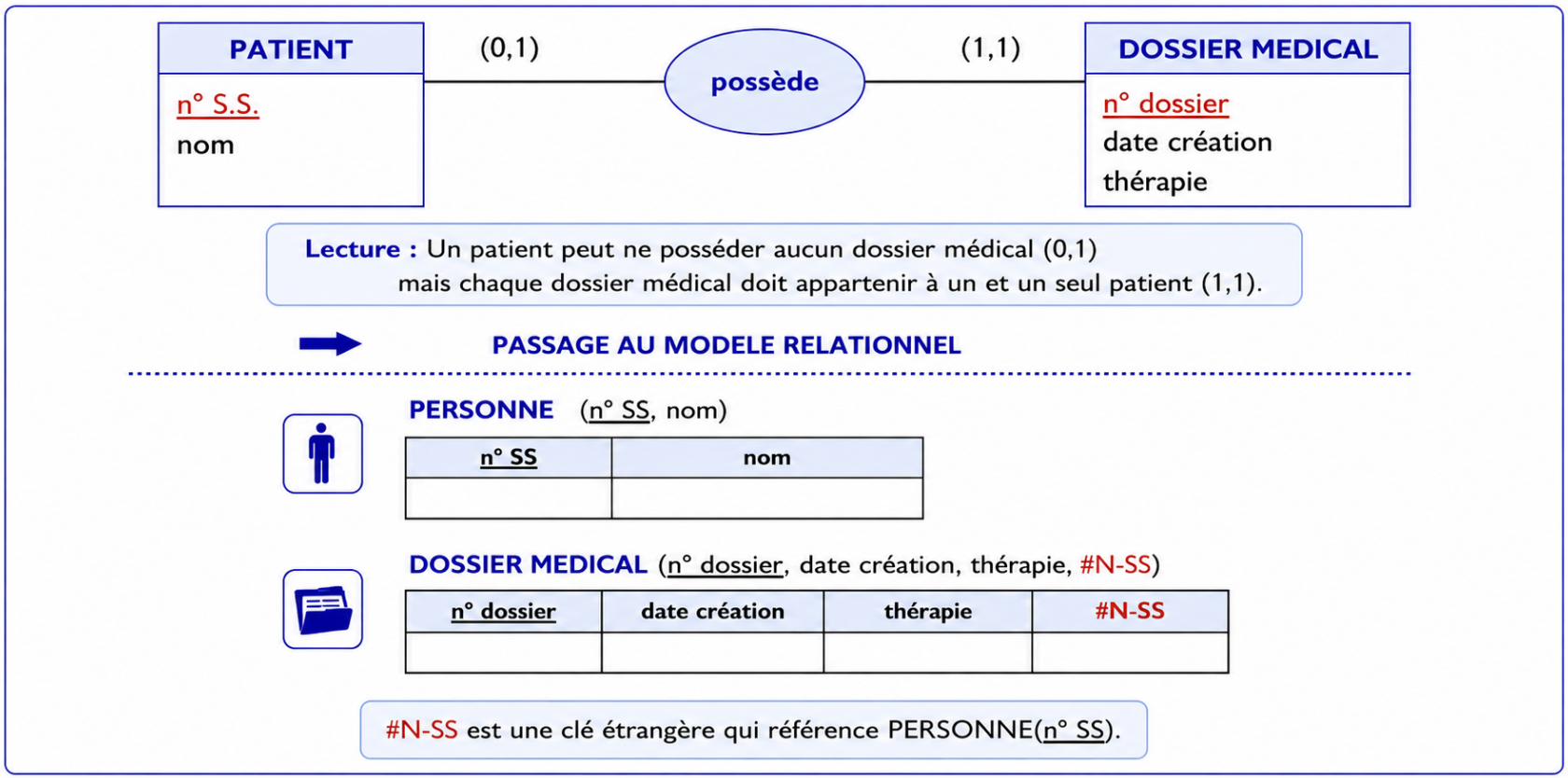
Passage du modèle E/A au modèle relationnel

- **Règle 4: Traduction des associations 1-1 (2/4)**
  - **Exemple 1**: les deux cardinalités minimales sont 1. On construit une seule table contenant les attributs des deux entités. La clé est reportée une seule fois si elle est la même.



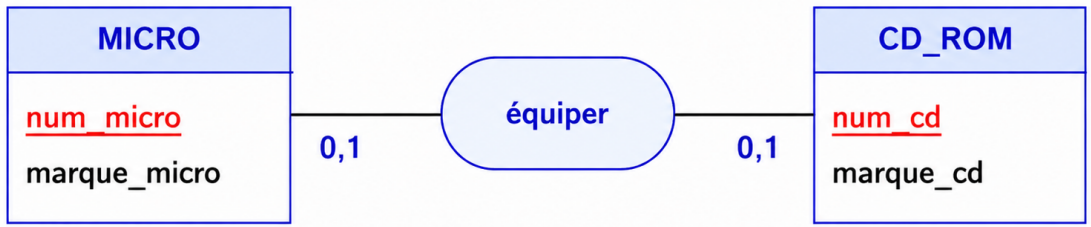
Passage du modèle E/A au modèle relationnel

- **Règle 4: Traduction des associations 1-1 (3/4)**
  - **Exemple 3**: une cardinalité minimale est 0, l'autre est 1. On reporte, dans l'entité dont la participation est totale, l'identifiant de l'autre entité.



Passage du modèle E/A au modèle relationnel)

- **Règle 4: Traduction des associations 1-1 (4/4)**
  - **Exemple 4**: les deux cardinalités minimales sont 0. On crée une table spécifique avec les identifiants des deux entités. Ce cas d'association nécessite la création d'une autre table (ici EQUIPER) prenant comme clé primaire la composition des clés primaires des autres tables devenant clé étrangère dans la table COMPOSER.



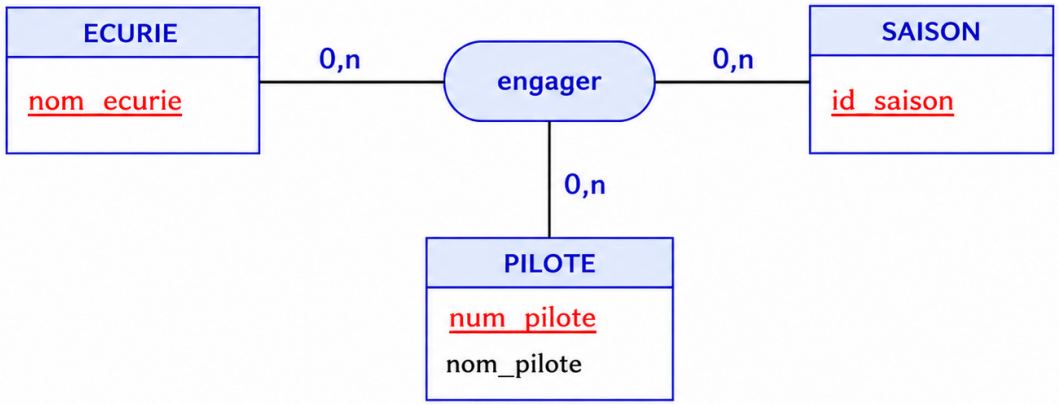
MICRO (Num_Micro, Marque_Micro)

CD_ROM (Num_Cd, Marque_Cd)

EQUIPER (Num_Micro#, Num_Cd#)

Passage du modèle E/A au modèle relationnel

- **Règle 5: Traduction des associations n-aires**
  - La table issue de l'association est composée des identifiants de toutes les entités participant à l'association comme clé étrangère et comme clé primaire composée. Les propriétés portées par l'association devenant des attributs de cette table.



SAISON (id_saison)

PILOTE (num_pilote , nom_pilote)

ECURIE (nom_ecurie)

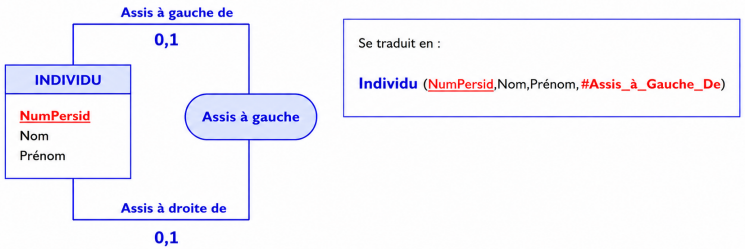
ENGAGER (id_saison# , nom_ecurie# , num_pilote#)

Passage du modèle E/A au modèle relationnel

- **Règle 6 : Traduction des associations récursives (1/2)**

- Une association réflexive $[1,1]$ sur une entité est traduite par une **clé étrangère** dans la relation représentant cette entité. Le nom de cette clé étrangère est celui de l'association.
- Une association réflexive $[1,N]$ est traduite par une **clé étrangère** dans la relation représentant cette entité.

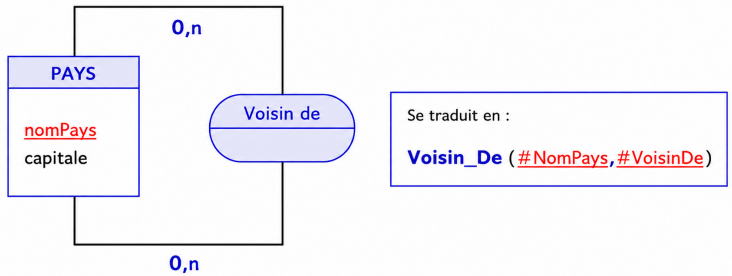
  Le nom de cette clé étrangère est :
  - celui de l'association si elle étiquette le trait de cardinalité maximale 1 ;
  - sinon, celui de l'association réciproque.



- **Dans les deux cas :** l'entité est d'abord traduite en relation, puis une clé étrangère est ajoutée dans cette même relation. Cette clé étrangère référence la clé primaire de la relation.

Passage du modèle E/A au modèle relationnel

- **Règle 6 : Traduction des associations récursives (2/2)**
  - Une association réflexive $[N, N]$ est traduite par une **nouvelle relation** portant le nom de l'association.
    - La relation représentant **l'entité d'origine est conservée**.
    - La nouvelle relation contient **deux clés étrangères** qui référencent toutes les deux la **clé primaire de la relation d'origine**.
    - Ces deux clés étrangères constituent ensemble la **clé primaire composée** de la nouvelle relation.



- Le schéma relationnel final contient donc deux relations :

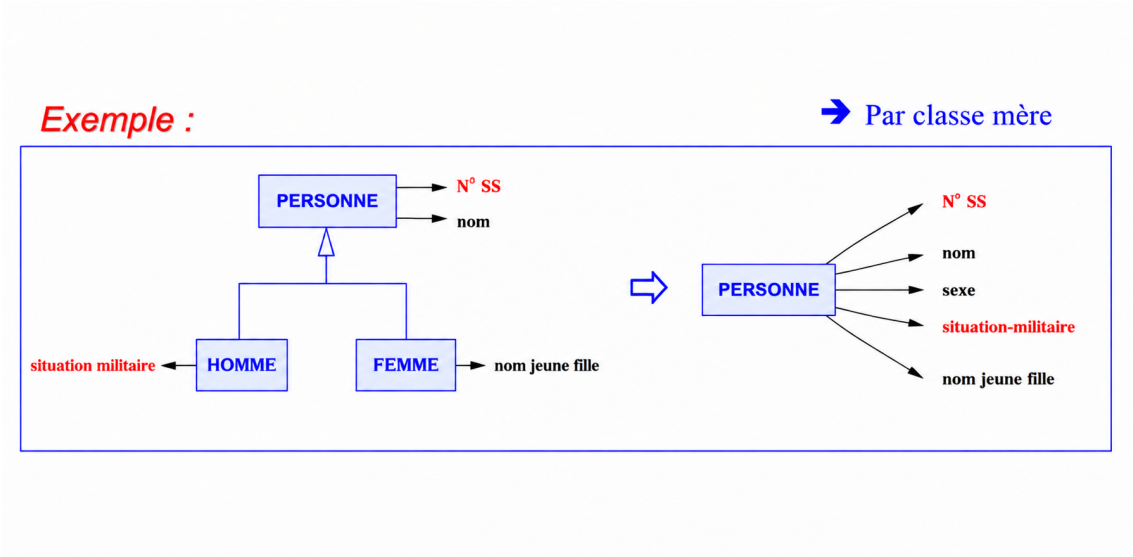
$$\text{PAYS}(\underline{nomPays}, capitale)$$
$$\text{VOISIN_DE}(\underline{\#NomPays,\ \#VoisinDe})$$

Passage du modèle E/A au modèle relationnel

- **Règle 7: traduction des liens de généralisation**
  - Les généralisations peuvent définir des couvertures totales ou partielles
    - TOTALE: tout élément est dans une sous-classe
      Exemple: PERSONNE en HOMME et FEMME
    - PARTIELLE: certaines éléments ne sont dans aucune sous-classe
      Exemple: PERSONNE en EMPLOYÉ et OUVRIER
  - Les généralisations peuvent définir des couvertures exclusives ou avec recouvrement
    - EXCLUSIVE: tout élément est au plus dans une sous-classe
      Exemple: PERSONNE en HOMME et FEMME
    - AVEC RECOUVREMENT ou IMBRIQUEE: certains éléments peuvent être dans plusieurs sous-classes
      Exemple: OISEAU en MIGRATEUR et VOLANT

Passage du modèle E/A au modèle relationnel

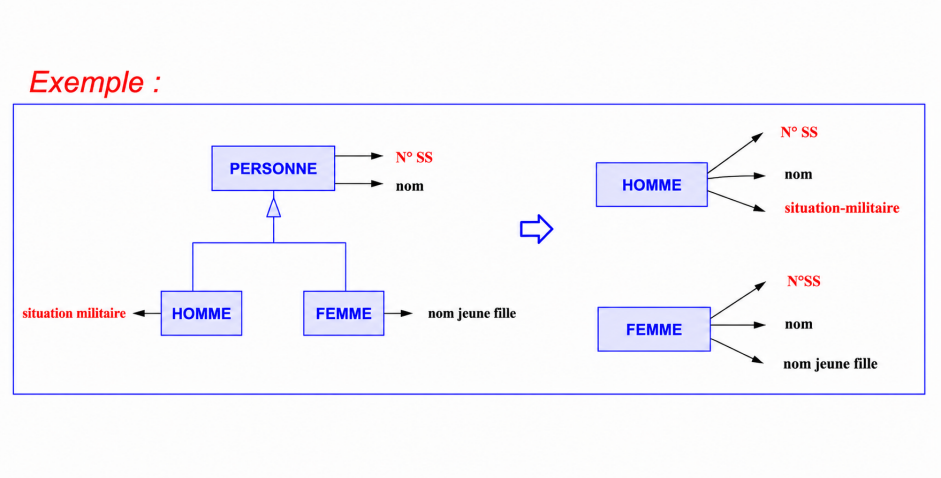
- **Règle 7: traduction des liens de généralisation**
  - Solution 1: Transformation de la hiérarchie de généralisation en une seule table



- **Avantages**:
- Solution la plus simple en nombre de relations
- Possible quelle que soit la hiérarchie (totale/partielle, exclusive/non exclusive)
- **Inconvénients**:
- Peut générer beaucoup d'attributs avec des valeurs nulles

Passage du modèle E/A au modèle relationnel

- **Règle 7: traduction des liens de généralisation**
  - Solution 2: Supprimer l'entité générique

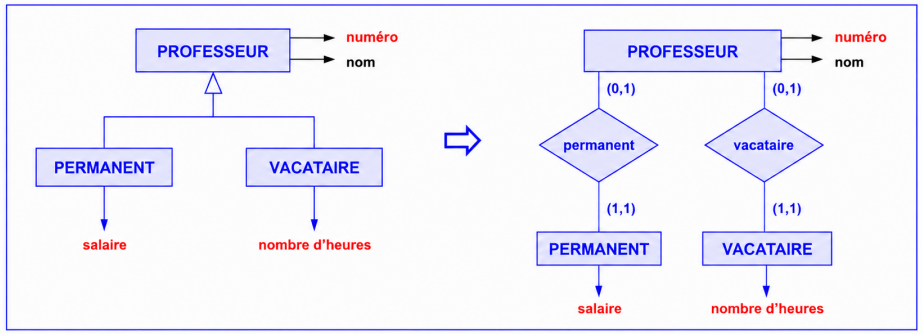


- **Avantages**:
- Intéressant quand le concept générique n'est pas important
- **Inconvénients**:
- Impossible quand la généralisation est partielle ou avec recouvrement (C'est pas obligatoire qu'une personne soit un ouvrier ou bien employer. Donc, on peut pas supprimer Personne).
- Perte de sémantique: notion de personne

Passage du modèle E/A au modèle relationnel

- **Règle 7: traduction des liens de généralisation**
  - Solution 3: Transformation de la hiérarchie des généralisations en associations

Exemple :



- **Avantages**:
- Possible quel que soit le type d'hiérarchie
- Pas de perte sémantique
- **Inconvénients**:
- Schéma résultant assez complexe

Passage du modèle E/A au modèle relationnel

Éléments de choix entre les trois transformations (solutions)

- La règle 2 est à rejeter si la hiérarchie n'est pas totale et exclusive.
- On considère les traitements (et leur fréquence):
  - Si les traitements utilisant de façon combinée les attributs génériques et spécifiques mais les spécifiques sont prépondérants, on peut supprimer la généralisation. On utilise donc la règle 2.
  - Si non, on utilise les règles 1 ou 3 selon les cas suivants:
    - Si les principaux traitements de mise à jour se font surtout sur l'ensemble des attributs (génériques et spécifiques), on utilise la règle 1 (Transformation de la hiérarchie de généralisation en une seule table).
    - Si non, on utilise la règle 3.

Conclusion

- Certaines règles sont automatisables.
  - Certains cas traitent cette traduction avec les règles standards.
  - Cependant, il y a de nombreux cas où le concepteur doit intervenir:
    - Choix entre différentes traductions.
    - Redéfinition des clés dans les relations N-aires.
    - Parfois, des considérations de traitement (niveau physique) sont anticipées.

Références

- Alan Beaulieu, Learning SQL, 3rd Edition O'Reilly Media, 2020, ISBN: 9781492057611.
- Lynn Beighley, Head First SQL: Your Brain on SQL – A Learner's Guide, 1st Edition O'Reilly Media, 2007, ISBN: 9780596526849.
- Cours R.Mghirbi, ENI Carthage, 2016-2017.
- Cours S.Cerrito, Université d'Evry val d'essonne, 2018 -2019.